

Group Project Proposal

Multi-Agent Cooperation and Adaptation in Competitive 3v3 Basketball Environment

Group ID: Group 16

DSA4213: Natural Language Processing for Data Science
Semester 1, AY2026/2027

Administrative Information

Group members	Ang Wei Hao, Damien (A0272596N) Hong Zheng Long Davin (A0273077Y) Koh Hong Yao (A0281127J) Peng Rui Biao (A0272075E) Ang Chin How, Xavier (A0271848R) Oh Rachel (A0282836U)
Project direction	LLM-based multi-agent cooperation and adaptive decision-making in a 3v3 Basketball Environment
Repository	https://github.com/ruibiaoo/llm-3v3-basketball-agents

Project Summary

This project investigates how structured teammate communication, captain-led tactical calls, and shared team memory affect cooperation and dynamic adaptation among Large Language Model (LLM) agents in competitive FIBA 3x3 basketball.

We propose a virtual half-court simulation featuring six autonomous player agents (three per team) and an LLM-based referee. Operating under partial observability, each player agent uses an LLM to generate tactical actions, natural-language teammate messages, and memory updates at key discrete game events. A deterministic non-LLM resolution engine enforces physics, rule constraints, while the LLM referee handles judgment-dependent contact fouls.

To evaluate these mechanisms, we conduct controlled ablation experiments comparing the full-coordination system against variants lacking teammate messaging, captain tactical calls, or dynamic

updates to shared team memory. Evaluation focuses on possession-level coordination metrics, scoring efficiency or points-per-possession (PPP), and counter-strategy adaptations within each game.

This project aims to demonstrate how communication, leadership, and memory retention shape multi-agent coordination, providing an explainable, scenario-analysis testbed for sports analytics.

1. Problem, Research Question, and Motivation

Research Question: How do structured teammate communication, captain-led tactical calls, and shared team memory affect cooperation and adaptation among LLM agents in a competitive 3v3 basketball environment?

The basketball environment provides a sequential, adversarial, and partially observable multi-agent problem. A player's decision depends not only on their individual ability, but also on teammate positioning, teammate intentions, defensive reactions, timing, memory, tactical strategy, and previous interactions. At the same time, this also makes it a useful environment for studying multi-agent coordination and adaptation because agents must make decisions while responding almost instantaneously to both teammates and opponents.

These dependencies are particularly important in FIBA 3x3 basketball. In FIBA 3x3, each team consists of three players and one substitute, and games are played on a half court with a 12-second shot clock (FIBA, n.d.). The short shot clock and continuous transitions create a fast-paced environment in which players must make decisions quickly while coordinating with teammates.

The intended and potential users are coaches and sports analysts who want to explore tactical scenarios before a game. Within our proposed system, we aim to allow coaches to provide natural-language player descriptions and pre-game strategies. For the purpose of this project, we will use publicly available player information from the FIBA 3x3 site to initialise the player agents. Whenever synthetic data is used, it will be stated. The simulator will not be presented as a validated predictor of real-world game outcomes. Instead, its initial purpose is to provide a controlled and explainable environment for comparing tactical scenarios and agent behaviours. For our project, we aim to utilise this interpretable system to study two identical teams (same player agents and setups), but with different constraints, to learn about communication between agents as well as agent adaptation.

The project will be considered successful if:

1. Agents consistently produce valid and basketball-relevant actions;
2. Removing teammate communication, captain-led tactical calls, or shared team memory updates produces measurable changes in team behaviour;
3. Agents demonstrate observable adaptation during the game with read and write access to shared team memory;
4. Coordination and decision-making can be analysed at the possession level; and
5. Observed differences between experimental conditions remain reasonably stable across different simulation seeds.

2. Related Work and Proposed Contribution

2.1 Multi-agent communication

Existing research has shown that communication can improve cooperation between LLM agents. For example, Madmoun and Lahlou (2026) found that introducing a simple communication channel substantially increased cooperation in a multi-agent coordination task. Research on multi-agent collaboration has also shown that factors such as agent governance, participation control, interaction patterns, and dialogue history can influence the quality and efficiency of collaborative decisions (Wang et al., 2025). In addition, research on LLM coordination suggests that agents can perform well when decisions depend on environmental information but may face greater difficulty when successful coordination requires reasoning about teammates' beliefs and intentions (Agashe et al., 2025).

Architecturally, Zhang et al. (2024) introduced CoELA, a LLM-based architecture that integrates perception, memory, communication, planning, and execution for decentralised cooperation in embodied environments. This agent architecture was tested in 2 environments, ThreeDWorld Multi-Agent Transport (TDW-MAT) and Communicative Watch-And-Help (C-WAH), and outperformed planning-based baselines, when being powered by GPT-4. CoELA was also found to be significantly better with memory. Complementing this architectural approach, Zhu et al. (2025) introduced MultiAgentBench, which examines how different centralised and decentralised coordination protocols, such as star and tree, and graph-mesh and chain respectively, and planning affects multi-agent teamwork, through milestone-based performance indicators. These studies motivate examining communication and team organisation as separate factors.

2.2 Multi-agent adaptation.

Effective cooperation and competition require agents to anticipate other agents' behaviour and adjust their decisions through interaction, which is essential when simulating a sport like Basketball. Yu et al. (2025) proposed Explicit Models of Opponents (EMO), which is essentially an inference-time prompting framework that maintains and continually updates an explicit model of each other agents based on their observed behaviour. This supports agent adaptation by using these evolving opponent models to adjust the agent's own reasoning and actions while other agents' strategies also change. In games such as Avalon, EMO combined with Chain-of-thought (COT) reasoning, improved the Assassin's accuracy in identifying Merlin from 60%, with COT only, to 91.7%. Mu et al. (2026) similarly studied Adaptive Theory of Mind (A-ToM), a prompt-based agent framework with an adaptation framework, that attempts to help an agent reason about what another agent may do, with the appropriate level of reasoning. Mu et al. (2026) argues that two agents, with the same level of reasoning of the possible actions of each other, may impair coordination, and their ability to work together as they will both be anticipating the other.

Shared team memory is another important factor in our proposed environment. Recent research has demonstrated that memory length can significantly alter collective and cooperative behaviour among LLM agents, although the effect can differ across models (Hishiki et al., 2026). This motivates our investigation of whether shared team memory can help agents adapt their decisions based on previous interactions with teammates and opponents during the game.

2.3 Multi-agent sports simulation.

Xu (2026) introduced somepark-football-agentic-simulator (sfas), a football simulation in which 22 LLM agents select high-level actions and a deterministic resolution engine advances the physical state. Each agent receives a shared, coarsened world map and acts without direct inter-agent messaging. Team identity is represented through low-rank adapters, while a knowledge graph supplies pre-match tactical settings. The study focuses on stable emergent play, including mechanisms to prevent possession monopolies and formation collapse. This provides a close architectural precedent for separating LLM decision-making from physical action resolution.

2.4 Proposed Contributions.

The reviewed studies motivate communication, team organisation, and memory as mechanisms for multi-agent coordination and adaptation, but do not directly study them together in a competitive 3v3 basketball setting. We will develop a 3v3 basketball environment to investigate how teammate

communication, captain-led tactical calls, and shared team memory updates affect cooperation and adaptation among LLM agents. Controlled ablations will isolate the effect of each mechanism, while possession-level analysis will measure changes in coordination, adaptation, and game performance. The intended contribution is an interpretable testing environment for studying multi-agent cooperation and adaptation in a competitive environment.

3. Agent and Task Setup

In our project, each player is represented as a persistent agent object. We simplify the environment such that each team owns three active player agents (two teams, 6 active agents in total) and no substitutes. At each decision step, the environment produces a structured observation for every agent, and each agent's turn proceeds through two layers:

1. **LLM decision layer**, responsible for reading the game state and deciding each agent's intent based on its own observations and its memory. The LLM's role is focused on the reasoning and coordination behaviour needed to choose the next action.
2. **Simulation resolution layer** (non-LLM), responsible for enforcing game rules and resolving physical and probabilistic action outcomes, and updating states. This includes enforcing movement limits, resolving the outcomes of passes, shots, and contests based on player abilities, actions and contextual factors, updating the score and clocks, and rejecting or safely remapping any illegal action an agent proposes.

Agent Observations

At each decision step, an agent receives:

1. Its own position on the grid and status, including possession and fatigue;
2. The position and movement of all other active players, including teammates and opponents, where movement is represented as a direction and speed;
3. Current defensive matchups (which defender is guarding which offensive player);
4. Ball position and possession (which player has the ball);
5. Score, game clock and shot clock, fouls;
6. The captain's current tactical call, when enabled, in natural language;
7. Recent teammate messages, when communication is enabled, in natural language;
8. Personal current-game memory containing relevant events from the ongoing game; and

9. Relevant shared team memory, when enabled, summarizing tendencies learned from previous occurred states, in natural language.

Players cannot receive the opposing team's captain calls or messages, and must therefore infer opponent intent from observable state alone. Refer to Code 1 in the appendix for an example on the observed state.

3.1 Player Profiles

Each player is assigned a profile that distinguishes its abilities and behavioural tendencies from other players. This is done by defining a player profile with the following attributes:

1. Abilities: shooting, passing, defense, stamina;
2. Playstyle: preferred moves, aggressiveness, passing and shooting tendency;

Attributes are given as a natural-language rating out of 10, provided by the coach, e.g. "Shooting: 6/10, Passing: 8/10...", while playstyle is a natural-language description of a player's tendencies, e.g. "Player tends to drive left and is good at cutting but has poor accuracy for under-basket shots..."

These attributes are included in the agent's profile and influence both its LLM-based tactical decisions and the simulation engine's resolution of action outcomes. Each agent also maintains their own personal current-game memory.

3.2 Team Strategy and Captain

One player on each team is designated captain. At each decision step, the captain may issue or maintain a tactical instruction in natural language to all teammates, which serves as the team's current tactical guidance. The captain may retain the previous call or issue a new one depending on how the game develops (see Section 4.3).

Teammates may use this call to guide their actions, or deviate from it when it is constrained or when their local observation recognises a better opportunity. Players may also communicate directly with teammates outside the captain's call (see Section 4.3). Non-captain players can also suggest tactics, but this will be primarily treated as a message.

4. Proposed System or Study Design

The technically meaningful component of the project is the agent coordination and adaptation framework. Rather than allowing LLMs to directly control the physical simulation, player agents operate through a structured decision interface that combines their individual profiles, observations, teammate communication, captain instructions and shared team memory.

The system separates three functions:

1. **LLM player agents** determine high-level tactical intent and teammate communication.
2. **The simulation engine** converts these intentions into legal actions and resolves the physical and stochastic outcomes.
3. **The referee agent** evaluates judgement-dependent rule events using the complete simulated game state.

This separation will allow the effects of LLM-based coordination to be studied, without allowing the LLM to directly determine the game outcomes.

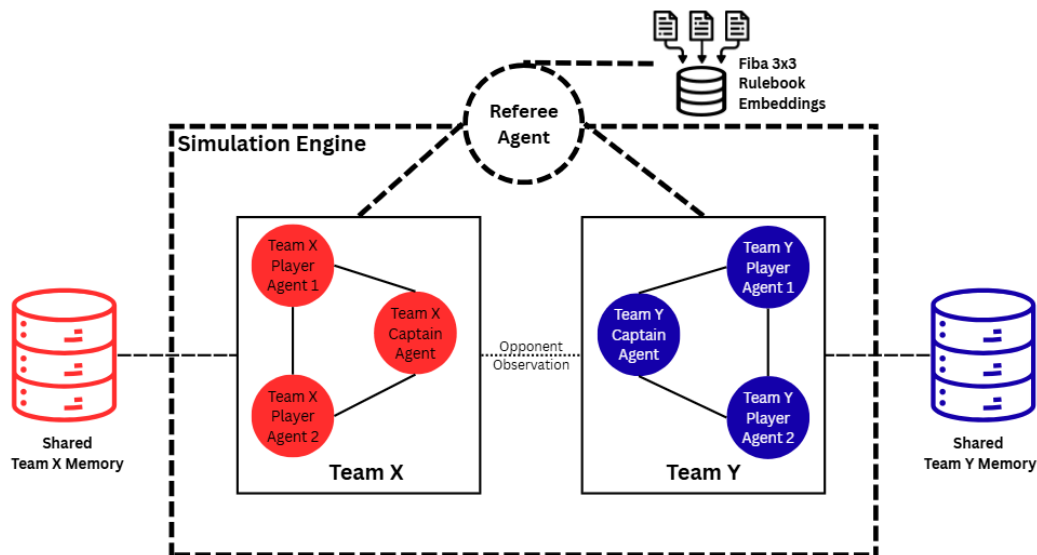


Figure 1. Proposed System Architecture

4.1 Overall Architecture

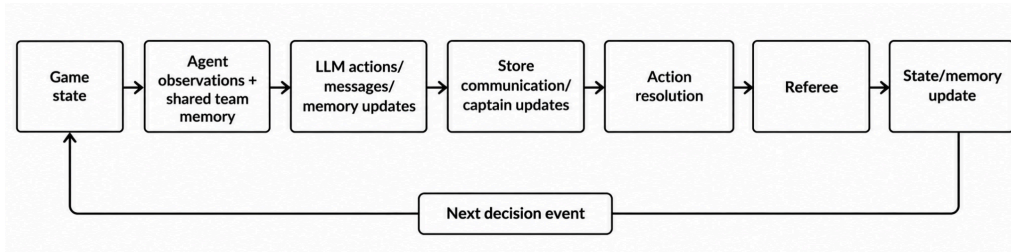


Figure 2. Main Pipeline for System

At each decision event, the simulator constructs an agent-specific observation from the current game state. Each player receives information about their own state, teammates, opponents, ball position, defensive matchups, score, clock, fouls, current captain's call, recent teammate messages and relevant opponent memory when the corresponding mechanisms are enabled. An example of state information can be found in Code 1 in the appendix. Code 2 also provides an example of how the player agent will receive the instructions.

The player will then produce two outputs:

- An action, such as but not limited to: *PASS*, *DRIVE*, *CUT_BASELINE*, *SCREEN*, *SHOOT* or *HELP_PAINT*
- A short natural-language communication message, which may communicate intentions or tactical information for teammates

The designated captain may additionally maintain the current tactical call or issue an updated tactical call.

Actions and communication messages are generated simultaneously at each decision event. Messages produced during the current decision event are stored by the communication channel and become available to teammates as part of their observations at the next decision event. Similarly, any updated captain tactical call becomes available to teammates at the next decision event. This allows communication and leadership to influence subsequent decisions without requiring an additional communication round within the same event.

The action is passed to the simulation engine, while the communication message and any updated captain call are stored for inclusion in teammates' observations at the next decision event. The engine then resolves interactions between simultaneous actions, applies physical constraints and updates the game state.

The resulting event and outcome are recorded in an event log and used to update their personal current-game memory and, when enabled, shared team memory.

4.2 Agent Decision and Simulation Cycle

Agents make decisions at meaningful game events rather than at every rendered frame. Events include a pass reception, screen engagement, drive, shot, rebound, turnover, defensive rotation, dead ball, or important shot-clock threshold. Each action consumes simulated time, allowing the game clock and shot clock to advance between decision events. All actions are then passed to the simulation engine to verify and resolve, an example set of actions are shown below:

```
""
<ACTIONS>
<A1 action="PASS" target="A2"/>
<A2 action="CUT_BASELINE" target="RIGHT_CORNER"/>
<A3 action="SCREEN" target="B2"/>

<B1 action="DEFEND" target="A1"/>
<B2 action="HELP_PAINT" target="A2"/>
<B3 action="DEFEND" target="A3"/>
</ACTIONS>
""
```

When shared team memory is enabled, each agent action will also include an optional memory reference identifying the specific stored memory item(s) used to inform that decision. If no stored memory is used, the reference is recorded as None. These references are stored in the event log to allow subsequent analysis of whether learned opponent information influences agent actions. For instance:

```
""
<ACTIONS>
<A1 action="PASS" target="A2" memory_ref="M03"/>
<A2 action="CUT_BASELINE" target="RIGHT_CORNER" memory_ref="None"/>
<A3 action="SCREEN" target="B2" memory_ref="M07"/>

<B1 action="DEFEND" target="A1" memory_ref="M12"/>
<B2 action="HELP_PAINT" target="A2" memory_ref="None"/>
<B3 action="DEFEND" target="A3" memory_ref="NONE"/>
</ACTIONS>
""
```

For each decision event:

1. The simulator generates six agent-specific observations;
2. Active players submit intended actions and, when enabled, communication messages;
3. The captain may maintain or update the team's tactical call;
4. Communication messages and updated captain calls are stored for the next decision event;
5. The engine resolves conflicts and action priority, beginning with the ball handler and primary defender, followed by off-ball interactions;
6. Proposed actions are checked for legality and physical feasibility;
7. Valid actions are resolved using the relevant player abilities and contextual game features;
8. The referee evaluates rule/judgment events;
9. Score, possession, clock, player states, memories and event logs are updated; and
10. The next meaningful decision event is triggered;

Player actions do not directly determine their outcomes. For example, a player selecting SHOOT does not guarantee a successful shot. Instead, the simulation calculates the outcome from player ability, contextual factors such as shot quality and defensive contest.

For example:

$$P(\text{make}) = \sigma(\beta_0 + \beta_1 \cdot \text{shooting} + \beta_2 \cdot \text{shotquality} - \beta_3 \cdot \text{contest} - \beta_4 \cdot \text{fatigue})$$

Analogous resolvers are used for passing, drives, turnovers, rebounds, screens and steals. Coefficients are kept explicit and can be subjected to sensitivity analysis rather than being implicitly determined by the LLM.

4.3 Communication, Leadership and Team Coordination

Communication and leadership are implemented as separate coordination mechanisms so that their effects can be independently evaluated.

Teammate communication. Players can send short natural-language messages to teammates in addition to their actions. Messages become visible to teammates at the following decision event and can therefore influence their subsequent actions. Messages will start with the sender's tag *<sender name>*, and end with a *</sender name>* tag. Messages are intended to communicate information such as movement

intentions, passing opportunities, defensive assignments, or tactical adjustments. Players can communicate and ask for certain actions, these messages will have an additional *[action]* tag.

The event logger will record:

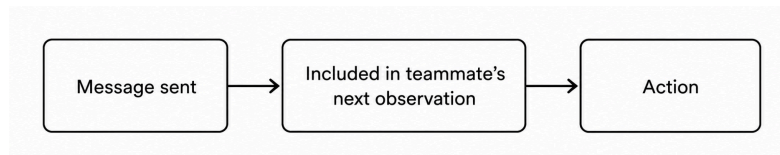


Figure 3. Message Flow Diagram

This will allow for possession-level analysis for whether communication translates to coordinated action.

Captain-led tactical calls. One player on each team is the designated captain. At each decision event, the captain may maintain the existing tactical call or issue an updated natural-language instruction. Any updated call becomes visible to teammates at the following decision event. The call is visible to teammates, but not to opponents. Messages by captains will start with a `<c>` tag and end with a `</c>` tag. Tactical calls will include an additional *[tactic]* tag in the message, with greater importance placed on it as compared to an *[action]* tag. Non-captain players can also suggest tactical calls, but will be treated as a message and tactical suggestion for the captain in the next state.

Players may follow the instruction or deviate when their local observation indicates a different action is better. This provides a controlled form of centralized guidance while retaining decentralized player decision-making. An example message received by a player agent is as such, here A1 is the captain:

```
"""  
<A1><c>[tactic cs=None] run pick and roll on the left wing, A2 will be handler  
[tactic]</c></A1>  
<A1><c>[action] A3 set a screen for A2 [/action]</c></A1>  
<A2>[action] A3 run pick and roll with me [/action]</A2>  
<A2>[tactic cs=None] I want to do pick and pop [tactic]</A2>  
<A2> be careful of opponent B1, good at stealing ball </A2>  
"""
```

4.4 Shared Team Memory

Each team maintains a shared team memory. At the beginning of each game, this memory is initialised with the team's existing understanding of its opponents, together with the team's game plan and any coach

instructions provided before the game. The shared team memory can be treated as a large collection of information available to the team, which also allows updates.

Memory schema. Within the shared team memory, shared opponent memory is stored as a structured, per-opponent record rather than raw text or an unbounded event log. Each entry records a unique memory ID, the opponent, a category (e.g. defensive tendency, shooting tendency, foul tendency), a short tendency description, a confidence value, the player who observed it, and the decision event at which it was recorded. This avoids the context bloating that an unstructured history would introduce. An example of the shared memory:

```
"""
[MEMORY ENTRY]
Memory ID: M03
Opponent: B1
Category: defensive_tendency
Tendency: "Leaves passing lanes open on contested drives"
Confidence: 1
Observed By: A1
Decision Event: evt_00031
"""
```

Write path. During the game, player agents can identify relevant opponent behaviours based on their observations and propose a memory update alongside their action. This proposal is not written directly to memory. The simulation checks that the update is well-formed and references an active opposing player, then compares it against existing entries for that opponent using similarity matching. If a similar tendency already exists, its confidence value is incremented and decision_event is updated. Otherwise, a new entry is created.

```
"""
[PROPOSED MEMORY UPDATE - Player A1]
<MEMORY_UPDATE opponent="B1" category="defensive_tendency" tendency="leaves passing lanes
open on contested drives" />
"""
```

```
"""
[VALIDATED MEMORY ENTRY]
Opponent: B1
Category: defensive_tendency
Tendency: "Leaves passing lanes open when contesting drives"
Confidence: 2 ← incremented (was 1)
"""
```

```
Observed By: A1
Decision Event: evt_00042 ← updated (was evt_00031)
"""
```

Read path. At each decision event, the player’s observation includes all current shared-memory entries relevant to opposing players, rendered in natural language (See Code 1).

Full-system and ablation conditions. In the full-system condition, agents can both read from and write to the shared team memory, allowing information observed by one player to contribute to the team’s understanding of its opponents and adapt its decisions within a game. In the memory ablation condition, the write-validation step is disabled. Proposed memory updates are parsed, but discarded rather than merged into the store. Agents retain read access to the initial memory, but cannot add new observations as the game progresses. If an agent refers to the memory to make a decision on the action, his action will include a memory reference. An example is shown below.

```
"""
<A1 action="PASS" target="A2" memory_ref="M03"/>
"""
```

The shared team memory is reset once the game ends, so knowledge is not carried over to new games.

Counter-Strategy (CS) Lookup. The player agents also maintain a counter strategy lookup table (with keys and values) as part of the shared team memory. The keys of the table will refer to the observed opponent’s strategy and the values will refer to the potential counter strategies. Strategies can be offensive or defensive by nature. The table starts off empty, and can be populated by the agents as the game starts. Whenever an agent references the counter strategy in their tactic messages, the tactic tag will include a “cs=key_id” parameter referencing the lookup table, as shown below:

```
"""
<A1><c>[tactic cs=01] run pick and roll on the left wing, A2 will be handler
[/tactic]</c></A1>
<A2>[tactic cs=None] A3 run pick and roll with me [/tactic]</A2>
"""
```

4.5 Referee Agent

Objective rule violations are handled directly by the simulation engine. These include events such as out-of-bounds violations, shot-clock violations, clock expiration, and other rule conditions that can be determined directly from the game state.

A separate LLM-based referee agent is used for judgement-dependent officiating events, such as determining whether physical contact constitutes a foul. To ground these judgements in the official ruleset rather than the LLM's implicit knowledge of basketball, the referee agent is paired with a Retrieval-Augmented Generation (RAG) mechanism over the FIBA 3x3 rulebook. Every game-state update and event information is passed to the referee agent for evaluation for fouls. Hence, the referee agent has access to the complete relevant game state for the judgement. An example of the state provided to the referee agent has been provided in Code 3 of the appendix.

Offline indexing. The FIBA 3x3 rulebook is split into chunks aligned to individual rule clauses (e.g. contact, technical or unsportsmanlike fouls), each embedded and stored in a vector store alongside a category label. This index is built once and shared across all experimental conditions and games.

Query construction and retrieval. After every state is resolved by the simulation engine, a query is built through fixed, deterministic templating over the structured contact fields found in Code 3 (e.g. contact type, offensive and defensive player states before and after contact, ball possession and contact location). This template-based approach, rather than free-text generation, keeps the query fully reproducible: the same event always produces the same query. The resulting query text is embedded and compared against the rulebook index. Candidates are first narrowed down using a rule-based category filter, and the top-3 most similar chunks within the category are retrieved

Referee decision. The retrieved rule excerpts are appended to the referee game state (Code 3) and passed to the referee LLM. The referee LLM outputs structured decisions as such:

```
"""
<REFEREE_DECISION
  foul="TRUE"
  offender="B1"
  fouled_player="A1"
  foul_type="DEFENSIVE_FOUL"
/>
"""
```

If the referee determines a certain action results in a foul, the action and foul are recorded and updated in the game state and passed to the players for the next game state.

```
"""
<REFEREE_DECISION
  foul="FALSE"
  offender="NONE"
/>
"""
```

```
fouled_player="NONE"
foul_type="NONE"
/>
"""
```

When the evaluated contact does not constitute a foul under the retrieved rules, the referee outputs the decision with foul = “FALSE” and null for offender/fouled_player/foul_type.

The referee therefore does not control objective game mechanics or player actions. The same referee model and configuration are used across all experimental conditions to keep officiating behaviour consistent between comparisons.

4.6 Comparisons and Ablations

The main experiments use four team configurations that differ only in the coordination mechanisms available to the agents. Player profiles, underlying LLM model and configuration, simulation parameters, and referee configuration are kept constant across conditions. All teams and agents retain their personal current-game memory, with the exception that writing to the shared team memory is removed for one of the teams. This allows the observed behavioural differences to be attributed more directly to the mechanism being investigated.

Configuration	Teammate Communication	Captain Calls	Shared Team Memory Updates
Team A - Full System	✓	✓	✓
Team B - No Communication	✗	✓	✓
Team C - No Captain	✓	✗	✓
Team D - Read-Only Shared Team Memory	✓	✓	✗ (Read-Only)

Table 1. Experimental Team Configurations for Ablation Study

The three primary comparisons are:

1. **Team A vs Team B:** How does teammate communication affect coordination?
2. **Team A vs Team C:** How do captain-led tactical calls affect coordination?
3. **Team A vs Team D:** How does the ability to update the shared team memory affect adaptation within games?

The full-system configuration (Team A) serves as the reference condition. For each ablation comparison, the full-system team and corresponding ablated team play directly against one another using identical player profiles and matched starting conditions.

5. Preliminary Evaluation Plan

The evaluation aims to determine whether teammate communication, captain-led tactical calls, and shared team memory updates produce measurable changes in multi-agent cooperation and adaptation. Evaluation is based on structured event logs recorded automatically during simulation. These logs capture agent actions, communication messages, captain calls, action outcomes, game-state transitions, referee decisions, and shared-memory updates. From these logs, we derive possession-level, and game-level metrics, aggregated across multiple runs, to evaluate coordination, adaptation, internal validity, and robustness across repeated runs.

The main evaluation uses controlled ablation experiments. The full system, Team A, will be compared against Team B without teammate communication, Team C without captain-led tactical calls, and Team D without the ability to update shared team memory. Player profiles, initial game conditions, team strategies, and simulation parameters will be kept constant across corresponding comparisons. Matched simulation seeds will be used where possible for corresponding comparisons. Results will be aggregated over multiple games and possessions rather than interpreted from individual outcomes.

Claim	Metrics	What it measures	Protocol	Expected evidence
Teammate communication improves cooperation	Points-Per-Possession (PPP)	Overall scoring efficiency	Team A vs Team B using matched conditions and repeated runs	Lower coordination conflicts and greater successful coordinated behaviour when communication is enabled
	Message-Action Alignment Rate (MAAR)	Alignment between teammate messages and subsequent actions		
Captain-led tactical calls improve team coordination	Points-Per-Possession (PPP)	Overall scoring efficiency	Team A vs Team C using matched conditions and repeated runs	Greater tactical coherence and successful execution of team-level actions when captain calls are available
	Captain-Call Adherence Rate (CCAR)	Adherence to captain instructions		
Shared Team memory improves adaptation within the same game	Points-Per-Possession (PPP)	Performance differences when Shared Team Memory updates are enabled	Team A vs Team D using matched initial memory, starting conditions and repeated runs	Increasing use of learned opponent tendencies later in the same game
	Counter-Strategy Selection Rate (CSSR)	Use of learned opponent information		
	Memory-Action Utilisation Rate (MAUR)	How frequent is the shared-memory used in decision making		

Simulation maintains valid actions and consistent game states	Invalid Action Rate (IAR)	Frequency of illegal, infeasible actions by agents	Automated checks across all experiments	Low invalid-state and action rates
	State Inconsistency Rate (SIR)	Frequency of impossible, contradictory game states during simulation		

Table 2. Evaluation Metrics for Coordination, Adaptation, and System Validity

5.1 Evaluation Phases

Phase 1: System Validation

Before collecting experimental results, pilot games will be used to verify that the simulator operates correctly. This includes checking action legality, state consistency, event logging, memory updates, referee triggering, and runtime/API usage. These pilot runs will be used for debugging and calibration and will not be included in the final experimental analysis.

Phase 2: Main ablation experiments

The three ablation comparisons will then be conducted:

- Team A vs Team B (Effect of teammate communication)
- Team A vs Team C (Effect of captain-led tactical calls)
- Team A vs Team D (Effect of shared-memory updates)

For each comparison, player profiles, starting conditions, team strategies, LLM settings, referee configuration, simulation parameters will be kept constant to isolate the effects of teammate communication, captain-led tactical calls, and shared team memory respectively.

Phase 3: Repeated-Run and Qualitative Analysis

The ablation experiments will be repeated across multiple simulation seeds to determine whether the observed behavioural differences remain reasonably consistent under stochastic variation. Representative possessions will also be manually reviewed to examine successful and failed coordination, use of communication and captain calls, and evidence of memory-driven adaptation alongside the aggregate quantitative metrics.

5.2 Metric Definitions

The metrics in Table 2 will be calculated from structured event logs generated during each simulation. Where possible, they will rely on automatically detectable game events to minimise subjective judgement.

Metrics related to communication, captain calls and shared memory updates will only be evaluated in the experimental conditions where these features are present.

5.2.1 Points-per-possession (PPP)

$$PPP = \frac{\text{Total points scored}}{\text{Total possessions}}$$

Points Per Possession (PPP) is a common basketball metric that measures the number of points scored by a team per possession (NBA, n.d.). In this study, a possession begins when a team gains control of the ball and ends when the opposing team gains control, following events such as a made basket, defensive rebound, steal, or turnover. An offensive rebound does not begin a new possession.

5.2.2 Message–Action Alignment Rate (MAAR)

$$MAAR = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(A_i \sim M_i)$$

where:

- N is the total number of messages with action tags,
- M_i is the teammate message with an action tag received before any decision,
- A_i is the action taken following message M_i ,
- $A_i \sim M_i$ indicates that action A_i is consistent with the intent of message M_i , and
- $\mathbb{I}(\cdot)$ is an indicator function that equals 1 if the condition is satisfied and 0 otherwise.

MAAR measures how consistently an agent’s actions align with messages received from its teammates with an *[action]* tag. Here, we define a message-conditioned decision as an action taken after receiving a teammate message with the *[action]* tag. An action is considered aligned if it is consistent with the intent communicated in the message. A higher MAAR therefore indicates that teammate communication is more consistently reflected in subsequent agent behaviour.

5.2.3 Captain-Call Adherence Rate (CCAR)

$$CCAR = \frac{N_{\text{actions consistent with captain messages}}}{N_{\text{actions given by captain messages}}}$$

Captain-Call Adherence Rate (CCAR) measures the proportion of actions that follow actions issued by the team captain, denoted by the *<c>[action]* tags. Here, we define an action as adhering to a captain's call if it is consistent with the action specified by the captain’s message. A higher CCAR indicates that agents more consistently follow captain-led decisions. Here, our setup will ensure that the actions issued by the captain closely aligns with the current tactic in play.

5.2.4 Counter-Strategy Selection Rate (CSSR)

$$CSSR = \frac{N_{\text{selected counter-strategies from lookup}}}{N_{\text{strategies}}}$$

Player agents are able to update their team's CS look up table, if they have write access to the shared team memory. Messages are mainly considered for this metric. Here, if the counter-strategy (marked with a *[tactic]* tag) is chosen from the CS table (denoted by the cs parameter in the tag), this is counted as a selected counter-strategy from lookup. The CSSR is the ratio of the count of all selected counter-strategies from lookup over the total number of strategy calls. A high CSSR means that agents are using the possible counter-strategy information they learnt, suggesting higher adaptability.

5.2.5 Memory-Action Utilisation Rate (MAUR)

$$MAUR = \frac{\text{Number of actions with memory reference not equal to None}}{\text{Total number of actions}}$$

MAUR measures how frequently agents make use of relevant opponent information stored in the shared team memory when making subsequent decisions. Each learned opponent tendency is stored as a memory item with a unique identifier. A decision is counted as memory utilisation when the agent retrieves and references a relevant memory item while selecting its action. We count the ratio of the number of actions where the memory reference is not None to the total number of actions. A higher MAUR therefore indicates that learned opponent information is more consistently incorporated into later decision-making.

5.2.6 Invalid Action Rate (IAR)

$$IAR = \frac{\text{Number of invalid actions}}{\text{Total number of proposed actions}}$$

IAR measures how frequently agents propose actions that are illegal, infeasible, or malformed given the current game state. An action is considered invalid if it cannot be executed under the simulation rules, such as attempting to pass without possession or selecting an unavailable action. A lower IAR therefore indicates that agents are more consistently producing valid actions that conform to the permitted action space.

5.2.7 State Inconsistency Rate (SIR)

$$SIR = \frac{\text{State transitions producing an inconsistent state}}{\text{Total post-resolution states checked}}$$

SIR measures how frequently the simulation engine produces an impossible or contradictory game state following a state transition. A state is considered inconsistent if it violates predefined game-state constraints, such as more than one player possessing the ball simultaneously. A lower SIR therefore indicates greater internal consistency and reliability of the simulation environment. A complete list of game states can be found in table 3 in the appendix.

6. Risks, Limitations, and Responsible Practice

6.1 Technical and operational risks

- **Invalid or malformed LLM actions.** Agents may propose infeasible actions (e.g. passing without possession) or produce badly formatted outputs.
 - *Safeguard:* All proposed actions pass through the non-LLM resolution layer, our simulation engine, which validates them against the game state. Illegal or malformed actions are rejected or remapped to a safe default (e.g. hold position or a safe pass) so the simulation never halts. The invalid-action rate (IAR) is reported as a system-validity metric (Section 5).
- **Cost, latency and rate limits.** Six player agents plus a referee agent across repeated games and seeds can consume substantial tokens and API calls.
 - *Safeguard:* LLM calls occur only at meaningful decision events (Section 4.2), not every frame. Pilot runs (Phase 1) will estimate cost and runtime before the main experiments. If quotas become binding, the number of games or seeds will be reduced before any condition is dropped, so that comparisons stay matched.

6.2 Experimental limitations

- **LLM non-determinism and attribution.** Outcome differences between conditions could come from sampling noise or stochastic game results rather than the mechanism being ablated.
 - *Safeguard:* The model, prompts, player profiles, referee and simulation parameters are held constant across Teams A to D. Matched seeds and repeated games are used, and no conclusion rests on a single game.
- **Model dependence.** Effects of communication, leadership or memory may be specific to the chosen LLM and prompts, since prior work reports that memory effects vary across models. Findings will be stated as applying to the tested configuration and will not be generalised to LLM agents broadly.
- **Simulation validity (sim-to-real gap).** The coarse grid, discrete events and hand-set resolution probabilities (e.g. the logistic shot model) cannot capture real basketball physics or footwork, and the coefficients are assumptions rather than fitted values. Results describe agent behaviour within this simulator, not real basketball outcomes. Coefficients are explicit so that sensitivity analysis can be run.

- **Referee inconsistency.** The LLM referee may make inconsistent foul judgements.
 - *Safeguard:* Objective violations (shot clock, out-of-bounds) are hard-coded in the engine. The referee handles only judgement-dependent contact events, sees the full game state, and uses a fixed low-variability configuration across all conditions. Referee decisions are logged.
- **Memory drift and contamination.** Agents may record inaccurate opponent observations in the shared team memory, and unbounded memory can bloat the context and mislead later decisions.
 - *Safeguard:* The shared team memory uses bounded, structured tendency summaries rather than raw event history. Every update is logged with the behaviour, responsible player and decision event. The read-only condition (Team D) separates the effect of learning from the effect of simply having initial opponent information.

6.3 Responsible practice

- **Intended use.** The simulator is a research testbed and an exploratory tool for comparing tactical scenarios. It is not a predictor of game results, a player-evaluation or selection tool, an automated officiating system, or a basis for betting or other high-stakes decisions.
- **Data and player profiles.** The project uses simulated data, publicly available FIBA 3x3 statistics and illustrative 1 to 10 ratings. No private, biometric or demographic data is used. Coach-provided descriptions are limited to basketball abilities and tendencies needed to initialise profiles, and they exclude personal or disparaging content.
- **Reproducibility.** Prompts, configurations, seeds and event logs will be shared in the repository. No API keys, proprietary tracking data or copyrighted match feeds will be included.

Appendix

```
""
[GAME STATE - PLAYER A1]

[OWN STATE]
Player: A1
Team: A
Position: (4, 6)
Has Ball: true
Fatigue: 2/10

[TEAMMATES]
A2:
  Position: (7, 4)
  Movement: CUTTING_LEFT
  Speed: FAST
A3:
  Position: (5, 8)
  Movement: TOWARDS_A1
  Speed: SLOW

[OPPONENTS]
B1:
  Position: (4, 7)
  Movement: TOWARDS_A1
  Speed: MEDIUM
B2:
  Position: (7, 5)
  Movement: FOLLOWING_A2
  Speed: FAST
B3:
  Position: (5, 9)
  Movement: TOWARDS_PAINT
  Speed: MEDIUM

[DEFENSIVE MATCHUPS]
B1 -> A1
B2 -> A2
B3 -> A3

[BALL]
Position: (4, 6)
Possession: A1

[GAME]
Score: Team A 8 - 6 Team B
```

```

Game Clock: 08:30
Shot Clock: 7
Team A Fouls: 1
Team B Fouls: 2

[PREVIOUS CAPTAIN CALL]
<A3><c>[tactic cs=None] Run pick and roll on the left wing. A1 is the ball handler.
[/tactic]</c></A3>

[PREVIOUS TEAMMATE MESSAGES]
<A2>[tactic cs=None] Lets do pick and pop. [/tactic]</A2>
<A2>[action] Look for me cutting to the basket. [/action]</A2>
<A3>[action] I will screen B1. [/action]</A3>

[SHARED TEAM MEMORY]
B1: leaves passing lanes open on contested drives (confidence: 2)
B2: foul prone (confidence: 1)

[/GAME STATE]
"""

```

Code 1. Example of game state observation for Player A

```

"""
[PLAYER PROFILE]
Shooting: 7/10
Passing: 8/10
Defense: 6/10
Stamina: 8/10

Playstyle:
Prefers driving to the basket and passing to open cutters.

[OBSERVATION]
...game state as shown in Code 1...

[INSTRUCTION]
Based on your player profile, current observation, teammate
communication, captain call and memory:

1. Select your next action.
2. Optionally send a short message to your teammates.
3. Optionally propose a shared team memory update.

Return only the required structured output.
"""

```

Code 2. Example Instruction for Player Agent

""

[REFEREE GAME STATE]

[GAME]

Score: Team A 8 - 6 Team B

Game Clock: 08:27

Shot Clock: 4

Possession: Team A

Ball Handler: A1

Team A Fouls: 1

Team B Fouls: 2

Ball Status: LIVE

[PLAYERS]

A1:

Team: A

Position: (5, 7)

Previous Position: (4, 6)

Movement: DRIVE_RIGHT

Speed: FAST

Has Ball: true

A2:

Team: A

Position: (7, 5)

Previous Position: (7, 4)

Movement: CUTTING_LEFT

Speed: FAST

Has Ball: false

A3:

Team: A

Position: (5, 8)

Previous Position: (5, 8)

Movement: HOLD

Speed: NONE

Has Ball: false

B1:

Team: B

Position: (5, 7)

Previous Position: (4, 7)

Movement: CONTEST_DRIVE

Speed: FAST

Has Ball: false

B2:

Team: B

Position: (7, 6)

Previous Position: (7, 5)
Movement: FOLLOWING_A2
Speed: MEDIUM
Has Ball: false

B3:

Team: B
Position: (5, 9)
Previous Position: (5, 9)
Movement: HELP_PAINT
Speed: MEDIUM
Has Ball: false

[DEFENSIVE MATCHUPS]

B1 -> A1

B2 -> A2

B3 -> A3

[BALL]

Position: (5, 7)

Possession: A1

[CURRENT EVENT]

Event Type: DRIVE

Offensive Player: A1

Primary Defender: B1

A1 Action:

DRIVE

Direction: RIGHT

Target: BASKET

B1 Action:

DEFEND

Target: A1

[CONTACT EVENT]

Contact Detected: true

Players Involved: A1, B1

Contact Location: (5, 7)

Contact Type: BODY_CONTACT

A1 State Before Contact:

Position: (4, 6)

Movement: DRIVE_RIGHT

Has Ball: true

B1 State Before Contact:

Position: (4, 7)

Movement: CONTEST_DRIVE

A1 State After Contact:

Position: (5, 7)

```

Movement: INTERRUPTED
Has Ball: true

B1 State After Contact:
  Position: (5, 7)
  Movement: HOLD

[/REFEREE GAME STATE]
"""

```

Code 3. Example Referee State

Game-State Constraint	Simple explanation	Example of an invalid state
1. Unique Ball Possession	At most one player can possess the ball at any time.	A1 and A2 both have has_ball = true.
2. Ball-Possessor Consistency	If a player has possession, the ball's state/location must agree with that player.	ball_holder = A1, but A1 is recorded as not possessing the ball.
3. Valid Player Positions	Every active player must remain within the valid half-court/grid boundaries.	A player has coordinates outside the court.
4. Valid Ball Position	The ball must be at a valid court location or in a valid transitional state, such as during a pass or shot.	The ball suddenly exists outside the simulated court with no corresponding event.
5. Valid Player Count	There should be exactly three active players per team during normal live play.	Team A has four active players on court.
6. Valid Team Membership	Every active player belongs to exactly one of the two teams.	A1 is simultaneously recorded as belonging to Team A and Team B.
7. Possession-Dependent Actions	Actions requiring possession can only be performed by the ball handler.	A2 attempts PASS or SHOOT while A1 has the ball.
8. Action Physical Feasibility	An action must be physically possible given the player's current position/state.	A player attempts a screen at a location they cannot reach.
9. Movement Limit	A player cannot move farther than the simulation allows during one action/event.	A player instantly moves from the paint to the opposite side of the court.

10. Game Clock Validity	The game clock cannot increase during normal play or become negative.	Clock changes from 08:30 to 08:35.
11. Shot Clock Validity	The shot clock must decrease appropriately and remain within its permitted range unless legitimately reset.	Shot clock changes from 5 seconds to 12 seconds without a reset event.
12. Shot-Clock Reset Consistency	The shot clock should only reset when a game event/rule requires it.	A normal pass resets the shot clock.
13. Possession Transition Consistency	Possession may change only following an event that can legitimately transfer control.	Team B suddenly gains possession without a steal, turnover, rebound, made basket, etc.
14. Score Consistency	The score can change only after a successfully resolved scoring event, and by the appropriate number of points.	Team A gains points after a missed shot.
15. Rebound Consistency	A rebound can occur only after a missed shot and must result in a valid ball-control state.	A player records a rebound without a preceding missed shot.
16. Turnover Consistency	A turnover must cause the offensive team to lose possession.	A1 commits a turnover but Team A remains the ball-controlling team.
17. Defensive Matchup Validity	Defensive assignments must reference currently active opposing players.	B1 is recorded as guarding their teammate B2.
18. Fatigue/Player-State Bounds	Player-state variables such as fatigue must remain within their predefined valid ranges.	Fatigue becomes -2 or exceeds its maximum value.
19. Foul-State Consistency	Fouls can only be added following a valid foul event/referee decision.	A player's foul count increases without any foul being called.
20. Dead-Ball Consistency	When the game is in a dead-ball state, normal live-ball actions should not execute until play resumes.	A player performs DRIVE while the ball is dead.

Table 3. List of Game-state constraints

References

Agashe, S., Fan, Y., Reyna, A., & Wang, X. E. (2025). LLM-coordination: Evaluating and analyzing multi-agent coordination abilities in large language models. In Findings of the Association for Computational Linguistics: NAACL 2025. Association for Computational Linguistics.

<https://aclanthology.org/2025.findings-naacl.448/>

FIBA. (n.d.). 3x3 basketball. Retrieved September 2026, from

<https://about.fiba.basketball/en/our-sport/3x3-basketball>

Hishiki, T., Arita, T., & Suzuki, R. (2026). How memory can affect collective and cooperative behaviors in an LLM-based social particle swarm (arXiv:2604.12250). arXiv. <https://arxiv.org/abs/2604.12250>

Madmoun, H., & Lahlou, S. (2026). Communication enables cooperation in LLM agents: A comparison with curriculum-based approaches. In Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2026). Association for Computational Linguistics.

<https://aclanthology.org/2026.eacl-short.23/>

Mu, C., Zeng, Y., Zhang, Q., Shao, K., Chu, C., Guo, H., Jia, D., Wang, Z., & Hu, S. (2026). Adaptive theory of mind for LLM-based multi-agent coordination. Proceedings of the AAAI Conference on Artificial Intelligence, 40(35). <https://doi.org/10.1609/aaai.v40i35.40204>

NBA. (n.d.). NBA advanced stats glossary. National Basketball Association. Retrieved September 2026, from <https://www.nba.com/stats/help/glossary>

Wang, H., Zhao, S., Wang, J., Qiang, Z., Qin, B., & Liu, T. (2025). Beyond frameworks: Unpacking collaboration strategies in multi-agent systems. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025). Association for Computational Linguistics.

<https://aclanthology.org/2025.acl-long.1037/>

Xu, Lingxiao (2026). someopark-football-agentic-simulator: A Multi-Agent LLM System for Football Simulation with Low-Rank Team Identity and Stabilised Emergent Possession (August 18, 2026).

<http://dx.doi.org/10.2139/ssrn.7098039>

Yu, X., Zhang, W., & Lu, Z. (2025). LLM-based explicit models of opponents for multi-agent games. In Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for

Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.naacl-long.41>

Zhang, H., Du, W., Shan, J., Zhou, Q., Du, Y., Tenenbaum, J. B., Shu, T., & Gan, C. (2024). Building cooperative embodied agents modularly with large language models (arXiv:2307.02485). arXiv. <https://arxiv.org/abs/2307.02485>

Zhu, K., Du, H., Hong, Z., Yang, X., Guo, S., Wang, Z., Wang, Z., Qian, C., Tang, X., Ji, H., & You, J. (2025). MultiAgentBench: Evaluating the collaboration and competition of LLM agents. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.acl-long.421>

AI Declaration

ChatGPT and other AI tools were used in refining the writing of this proposal and for idea generation. Figures 2 and 3 were also generated by ChatGPT. We are responsible for the quality of this proposal.